

c0a2515415 / ProjExD_Group07

<> Code

Issues

Pull requests

Actions

Projects

Wiki

Security and quality

Insights

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

base: 18ca61060c4faeca11f4ebea439...

...

compare: fdbbcc412a7c846ab9e025cff1d...

...

3 commits

1 file changed

1 contributor

Commits on May 19, 2026

- ボス実装と攻撃実装は完了

c0a2515415 committed 1 hour ago

9c144fc

<>

ボス機能実装完了

c0a2515415 committed 51 minutes ago

51e33bc

<>

ボス機能最終実装完了

c0a2515415 committed 16 minutes ago

fdbbcc4

<>

Showing 1 changed file with 134 additions and 105 deletions.

Split

Unified

239 rush_kokaton.py			
164	# self.image = self.font.render(f"Score: {self.value}", 0, self.color)	164	# self.image = self.font.render(f"Score: {self.value}", 0, self.color)
165	# screen.blit(self.image, self.rect)	165	# screen.blit(self.image, self.rect)
166		166	
167	-	167	+ # class Laser(pg.sprite.Sprite):
168	- # class Bomb(pg.sprite.Sprite):		
169	# """	168	# """
170	# 爆弾に関するクラス	169	# 爆弾に関するクラス
171	# 今回はボスが出してくる攻撃などとして使用	170	# 今回はボスが出してくる攻撃などとして使用
172	# """	171	# """
173	- # colors = [(255, 0, 0), (0, 255, 0), (0, 0, 255), (255, 255, 0), (255, 0, 255), (0, 255, 255)]		
174		172	
175	- # def __init__(self, emy: "Enemy", bird: Bird):	173	+ # def __init__(self, boss: "Boss", bird: Bird):
176	# """	174	# """
177	# 爆弾円Surfaceを生成する	175	# 爆弾円Surfaceを生成する
178	# 引数1 emy: 爆弾を投下する敵機	176	# 引数1 emy: 爆弾を投下する敵機
179	# 引数2 bird: 攻撃対象のこうかとん	177	# 引数2 bird: 攻撃対象のこうかとん
180	# """	178	# """
181	# super().__init__()	179	# super().__init__()
182	- # rad = random.randint(10, 50) # 爆弾 円の半径: 10以上50以下の乱数	180	+

https://github.com/c0a2515415/ProjExD_Group07/compare/18ca61060c4faeca11f4ebea4394fde9323272f5...fdbbcc412a7c846ab9e025cff1d27f6... 1/7

183	- # self.image = pg.Surface((2*rad, 2*rad))		
184	- # color = random.choice(__class__.colors) # 爆弾円の色 : クラス変数からランダム選択		
185	- # pg.draw.circle(self.image, color, (rad, rad), rad)		
186	- # self.image.set_colorkey((0, 0, 0))		
187	- # self.rect = self.image.get_rect()		
188	# # 爆弾を投下するemyから見た攻撃対象の birdの方向を計算	181	# # 爆弾を投下するemyから見た攻撃対象の birdの方向を計算
189	- # self.vx, self.vy = calc_orientation(emy.rect, bird.rect)	182	+ # self.vx, self.vy = calc_orientation(boss.rect, bird.rect)
190	- # self.rect.centerx = emy.rect.centerx	183	+ # self.rect.centerx = boss.rect.centerx
191	- # self.rect.centery = emy.rect.centery+emy.rect.height//2	184	+ # self.rect.centery = boss.rect.centery+boss.rect.height//2
192	# self.speed = 6	185	# self.speed = 6
193		186	
194	# def update(self):	187	# def update(self):
201	# self.kill()	194	# self.kill()
202		195	
203		196	
		197	+
204	# class Beam(pg.sprite.Sprite):	198	# class Beam(pg.sprite.Sprite):
205	# """	199	# """
206	# ビームに関するクラス	200	# ビームに関するクラス
264	self.kill()	258	self.kill()
265		259	
266		260	
267	- # class Enemy(pg.sprite.Sprite):	261	+ class Boss():
268	- # """	262	+ """
269	- # 敵機に関するクラス	263	+ ボスのクラス
270	- # ボスのクラス (大きさだけいじればいいかな)	264	+ 出現位置から降下させることで、ボス感を出している
271	- # """	265	+ インターバルはボスのレーザーの判定に使う (今回は違う)
272	- # imgs = [pg.image.load(f"fig/alien{i}.png") for i in range(1, 4)]	266	+ """
273		267	
274	- # def __init__(self):	268	+ def __init__(self):
275	- # super().__init__()	269	+ super().__init__()
276	- # self.image = pg.transform.rotozoom(random.choice(__class__.imgs), 0, 0.8)	270	+ # self.image = pg.transform.rotozoom(random.choice(__class__.imgs), 0, 0.8)
277	- # self.rect = self.image.get_rect()	271	+ self.image = pg.image.load("fig/alien1.png")
278	- # self.rect.center = random.randint(0, WIDTH), 0	272	+ self.image = pg.transform.scale(self.image, (300, 300))
279	- # self.vx, self.vy = 0, +6	273	+ self.rect = self.image.get_rect()
280	- # self.bound = random.randint(50, HEIGHT//2) # 停止位置	274	+ self.rect.center = (900, 300)
281	- # self.state = "down" # 降下状態or停止状態	275	+ self.vx, self.vy = 0, +6
282	- # self.interval = random.randint(50, 300) # 爆弾投下インターバル	276	+ self.bound = 330 # 停止位置

		277	+	<code>self.state = "down"</code> # 降下状態or停止状態
283		278		
284	- # <code>def update(self):</code>	279	+	<code>self.interval = random.randint(50,</code>
285	- # <code>"""</code>	280	+	<code>300)</code> # 爆弾投下インターバル
286	- # 敵機を速度ベクトル <code>self.vy</code> に基づき移動 (降下) させる	281	+	<code>def update(self, screen):</code>
287	- # ランダムに決めた停止位置 <code>_bound</code> まで降下 したら, <code>_state</code> を停止状態に変更する	282	+	<code>"""</code>
288	- # 引数 <code>screen</code> : 画面Surface	283	+	敵機を速度ベクトル <code>self.vy</code> に基づき移動 (降 下) させる
289	- # <code>"""</code>	284	+	ランダムに決めた停止位置 <code>_bound</code> まで降下し たら, <code>_state</code> を停止状態に変更する
290	- # <code>if self.rect.centery > self.bound:</code>	285	+	引数 <code>screen</code> : 画面Surface
291	- # <code>self.vy = 0</code>	286	+	<code>"""</code>
292	- # <code>self.state = "stop"</code>	287	+	<code>if self.rect.centery > self.bound:</code>
293	- # <code>self.rect.move_ip(self.vx, self.vy)</code>	288	+	<code>self.vy = 0</code>
		289	+	<code>self.state = "stop"</code>
		290	+	<code>self.rect.move_ip(self.vx, self.vy)</code>
		291	+	
		292	+	<code>screen.blit(self.image, self.rect)</code>
		293	+	
		294	+	
		295	+	<code>class Bomb(pg.sprite.Sprite):</code>
		296	+	<code>"""</code>
		297	+	ボスの攻撃の一つ
		298	+	爆弾円を生成し、爆破場所を知らせたのち、爆発画 像に切り替える
		299	+	衝突判定は爆発画像から
		300	+	生成場所は、こうかとんが動ける範囲の中でランダ ムな場所
		301	+	(これは今回はマージできてないので使用できな い)
		302	+	爆発中かどうかのステータス: <code>bom_status</code>
		303	+	により衝突判定を切り替える
		304	+	爆発までの時間: <code>bom_timer</code>
		305	+	<code>"""</code>
		306	+	<code>def __init__(self, boss: "Boss", bird:</code> <code>Bird, triple = 0):</code>
		307	+	<code>"""</code>
		308	+	引数: <code>triple</code> は、爆弾を三つ生成する際に使用 するだけのため
		309	+	初期値を0とし、爆弾を一つ生成する場合には 影響を与えない
		310	+	<code>"""</code>
		311	+	<code>super().__init__()</code>
		312	+	<code>rad = 40</code> # 爆弾円の半径
		313	+	<code>self.image = pg.Surface((2*rad,</code> <code>2*rad))</code>
		314	+	<code>color = (255, 0, 0)</code> # 爆弾円の色
		315	+	<code>pg.draw.circle(self.image, color,</code> <code>(rad, rad), rad)</code>
		316	+	<code>self.image.set_colorkey((0, 0, 0))</code>
		317	+	<code>self.rect = self.image.get_rect()</code>
		318	+	
		319	+	<code>self.rect.center = bird.rect.center</code>
		320	+	

		<pre> 321 + #こうかとんが動ける範囲をランダムに爆発す る 322 + self.rect.center = (323 + random.randint(50, 350) + triple, 324 + random.randint(50, GROUND + 200) 325 +) 326 + 327 + self.bom_timer = 0 328 + self.bom_status = "normal" 329 + 330 + def update(self): 331 + #爆弾円が表示されて、数秒経ったら爆発画像 に切り替える 332 + 333 + self.bom_timer += 1 334 + if self.bom_timer >= 200 and self.bom_status == "normal": 335 + 336 + center = self.rect.center 337 + 338 + self.image = pg.image.load(f"fig/explosion.gif") 339 + self.rect = self.image.get_rect() 340 + self.rect.center = center 341 + 342 + self.bom_status = "explosion" 343 + 344 + if self.bom_timer >= 250: 345 + self.bom_status == "normal" 346 + self.kill() </pre>	
294		347	
295		348	
296	<pre>class Life():</pre>	349	<pre>class Life():</pre>
314	<pre> screen.blit(self.img, (x, y))</pre>	367	<pre> screen.blit(self.img, (x, y))</pre>
315		368	
316		369	
317	<pre>- # class Gravity(pg.sprite.Sprite):</pre>		
318	<pre>- # """</pre>		
319	<pre>- # 今回は、小さい敵と障害物と爆弾を消せる必殺 技クラス</pre>		
320	<pre>- # """</pre>		
321	<pre>- # def __init__(self, life):</pre>		
322	<pre>- # super().__init__() #Groupのため に継承</pre>		
323	<pre>-</pre>		
324	<pre>- # self.life = life</pre>		
325	<pre>- # self.image = pg.Surface((WIDTH, HEIGHT))</pre>		
326	<pre>- # self.image.set_alpha(128)</pre>		
327	<pre>- # self.rect = self.image.get_rect()</pre>		
328	<pre>-</pre>		
329	<pre>- # def update(self):</pre>		
330	<pre>- # self.life -= 1</pre>		
331	<pre>- # if self.life <= 0:</pre>		
332	<pre>- # self.kill()</pre>		
333	<pre>-</pre>		
334	<pre>-</pre>		
335	<pre>- # class EMP():</pre>		

```

336 - #     def __init__(self, emys, bombs,
      screen):
337 - #         for emy in emys:
338 - #             emy.interval = float("inf")
339 - #             emy.image =
      pg.transform.laplacian(emy.image)
340 -
341 - #         for bom in bombs:
342 - #             bom.speed /= 2
343 - #             bom.state = "inactive"
344 -
345 - #         image = pg.Surface((WIDTH, HEIGHT))
346 - #         image.fill((255, 255, 0))
347 - #         image.set_alpha(200)
348 -
349 - #         screen.blit(image, [0, 0])
350 - #         pg.display.update()
351 - #         time.sleep(0.05)
352 -
353 -
354 - # class Shield(pg.sprite.Sprite):
355 - #     def __init__(self, life, bird:Bird):
356 - #         super().__init__()
357 - #         self.life = life
358 -
359 - #         self.image = pg.Surface((20,
      bird.rect.height * 2))
360 - #         pg.draw.rect(self.image, (0, 0,
      255), (0, 0, 10, bird.rect.height * 2))
361 - #         self.rect = self.image.get_rect()
362 -
363 - #         vx, vy = bird.dire #向いてる方向のベ
      クトルをvx, vyに代入
364 - #         self.rect.centerx =
      bird.rect.centerx + vx * bird.rect.width
365 - #         self.rect.centery =
      bird.rect.centery + vy * bird.rect.height
366 - #         kakudo = math.degrees(math.atan2(-
      vy, vx))
367 - #         self.image =
      pg.transform.rotozoom(self.image, kakudo, 1)
368 - #         self.image.set_colorkey((0, 0, 0))
      #ここに入れないと黒くなる
369 -
370 - #     def update(self):
371 - #         self.life -= 1
372 - #         if self.life <= 0:
373 - #             self.kill()
374 -
375 -

```

```

376 def main():
377     pg.display.set_caption("走れ！こうかとん")
378     screen = pg.display.set_mode((1100, 650))
385
386     # score = Score()
387     bird = Bird()

```

388

```

370 def main():
371     pg.display.set_caption("走れ！こうかとん")
372     screen = pg.display.set_mode((1100, 650))
379
380     # score = Score()
381     bird = Bird()
382 + boss = Boss()

```

383

389	-	# bombs = pg.sprite.Group()	384	+	bombs = pg.sprite.Group()
390		# beams = pg.sprite.Group()	385		# beams = pg.sprite.Group()
391		exps = pg.sprite.Group()	386		exps = pg.sprite.Group()
392		# emys = pg.sprite.Group()	387		# emys = pg.sprite.Group()
393		obstacle = pg.sprite.Group()	388		obstacle = pg.sprite.Group()
394			389		
395		life = Life(5)	390		life = Life(5)
			391	+	attack_count = 0
			392	+	bomb_cooldown = 0
396		# gravity = pg.sprite.Group() #課題2 Groupにインスタンスを追加	393		# gravity = pg.sprite.Group() #課題2 Groupにインスタンスを追加
397		# shield = pg.sprite.Group()	394		# shield = pg.sprite.Group()
398		while True:	395		while True:
399			396		
			397	+	bomb_cooldown += 1
			398	+	
			399	+	
			400	+	if tmr >= 6100:
			401	+	
			402	+	#爆弾が画面上になければ、生成する
			403	+	if len(bombs) == 0 and
					bomb_cooldown >= 200 and attack_count < 3:
			404	+	bombs.add(Bomb(boss, bird))
			405	+	bomb_cooldown = 0
			406	+	attack_count += 1
			407	+	
			408	+	#bombの処理 爆発を三回させるかどうか
			409	+	if len(bombs) == 0 and
					attack_count >= 3:
			410	+	#重なってしまうので、ずれた場所に生成する
			411	+	bombs.add(Bomb(boss, bird,
					-50))
			412	+	bombs.add(Bomb(boss, bird,
					0))
			413	+	bombs.add(Bomb(boss, bird,
					50))
			414	+	attack_count = 0
400			415		
401		#障害物	416		#障害物
402		if tmr % 60 == 0:	417		if tmr % 60 == 0:
506		# if tmr%200 == 0: # 200フレームに1回, 敵機を出現させる	521		# if tmr%200 == 0: # 200フレームに1回, 敵機を出現させる
507		# emys.add(Enemy())	522		# emys.add(Enemy())
508			523		
509	-	# for emy in emys:	524	+	# if boss.state == "stop" and
					tmr%boss.interval == 0:
510	-	# if emy.state == "stop" and	525	+	# # 敵機が停止状態に入ったら,
		tmr%emy.interval == 0:			intervalに応じて爆弾投下
511	-	# # 敵機が停止状態に入ったら,	526	+	# bombs.add(Bomb(boss, bird))
		intervalに応じて爆弾投下			
512	-	# bombs.add(Bomb(emy, bird))	527	+	
513			528		
514		# for emy in	529		# for emy in
		pg.sprite.groupcollide(emys, beams, True,			pg.sprite.groupcollide(emys, beams, True,
		True).keys(): # ビームと衝突した敵機リスト			True).keys(): # ビームと衝突した敵機リスト

515	# exps.add(Explosion(emy, 100))	530	# exps.add(Explosion(emy, 100))
	# 爆発エフェクト		# 爆発エフェクト
539	# for bomb in	554	# for bomb in
	pg.sprite.groupcollide(bombs, gravity, True, False):		pg.sprite.groupcollide(bombs, gravity, True, False):
540	# exps.add(Explosion(bomb, 50))	555	# exps.add(Explosion(bomb, 50))
541		556	
542	- # for bomb in	557	+ #グループからbombを取り出して、 bom_statusが
	pg.sprite.groupcollide(shield, bombs, True, True): #課題5 防御壁		
543	- # exps.add(Explosion(bomb, 50))	558	+ #"explosion"の時だけ衝突判定させる
544	-	559	+ for bomb in bombs:
		560	+ if bomb.bom_status ==
			"explosion":
		561	+ for bomb in
			pg.sprite.spritecollide(bird, bombs, True):
		562	+ exps.add(Explosion(bomb,
			50))
		563	+ life.num -= 1
		564	+ pg.display.update()
		565	+ if life.num <= 0:
		566	+ time.sleep(2)
		567	+ return
		568	+
545	for obstacles in	569	for obstacles in
	pg.sprite.spritecollide(bird, obstacle, True):		pg.sprite.spritecollide(bird, obstacle, True):
546	exps.add(Explosion(obstacles, 50))	570	exps.add(Explosion(obstacles, 50))
547	#こうかтон悲しみエフェクト	571	#こうかтон悲しみエフェクト
557	maps.update(screen, tmr)	581	maps.update(screen, tmr)
558		582	
559	bird.update(screen)	583	bird.update(screen)
		584	+ if tmr >= 6000:
		585	+ boss.update(screen)
560		586	
561	# beams.update()	587	# beams.update()
562	# beams.draw(screen)	588	# beams.draw(screen)
572	obstacle.update()	598	obstacle.update()
573	obstacle.draw(screen)	599	obstacle.draw(screen)
574		600	
		601	+ bombs.update()
		602	+ bombs.draw(screen)
		603	+
575	pg.display.update()	604	pg.display.update()
576	tmr += 1	605	tmr += 1
577	clock.tick(50) #FPSはこれ フレーム数。	606	clock.tick(50) #FPSはこれ フレーム数。
	50フレームで1秒を表すということ		50フレームで1秒を表すということ